# Chapter Five: Geometrical Transformations

## COMPUTER GRAPHICS

**COURSE NUMBER**: COSC3072

**PREREQUISITE:** COMPUTER PROGRAMMING (COSC 1012)

*Compiled by: Kidane W.*

# Introduction



- **Transformation** means changing some graphics into something else by applying rules.
  - Operations that alter the position, size, and orientation of geometric objects in a coordinate system.
- **Transformations** play an important role in computer graphics to reposition the graphics on the screen and change their size or orientation.
  - **Importance**: Essential for modeling, animation, and rendering in computer graphics.
  - **Applications**: Video games (moving characters), simulations (object interactions), CAD (designing).

# Homogeneous Coordinates



## What are Homogeneous Coordinates in Computer Graphics (with OpenGL)?

- Imagine you're playing a video game or using a drawing app. The computer needs to know where to put things like characters, shapes, or objects on the screen.
- Usually, we use (x, y, z) to describe positions in 3D - like saying: "The ball is 5 steps right (x), 3 steps up (y), and 2 steps forward (z)."
- That's pretty clear.
- But in computer graphics, especially with OpenGL, we use something a little different called: **Homogeneous Coordinates**: (x, y, z, w)

**We add one more number: w. It turns 3D positions into 4D: (x, y, z, w).**

### Why Add the Extra Number?

**1. Easier Math for Transformations**

- We often move, rotate, or scale things in games and graphics.
- Doing this math is MUCH easier and cleaner when we use **homogeneous coordinates**.
- With the extra **w**, we can use **matrix math** to do everything in one step!

**2. Perspective (Far Looks Smaller)**

- You know how faraway things look smaller?
- That magic is done using the **w** value.
- Homogeneous coordinates help make things look realistic by doing **perspective projection** - like turning 3D scenes into 2D screen views.

Homogeneous coordinates make **transformations like translation, rotation, scaling, and perspective** happen using **only matrix multiplication** — which computers are super fast at doing.

# Homogenous Coordinates



The **Cartesian coordinate system** is a coordinate system that uses two or more perpendicular axes to specify the location of a point in space. The point where the x-axis and y-axis intersect is called the **origin** (denoted as (0,0)).

Think of **Cartesian coordinates** like a grid or map that tells you **exactly where something is** in space.

- To perform a sequence of transformation such as translation followed by rotation and scaling, we need to follow a sequential process.
  1. **Translate the coordinates,**
  2. **Rotate the translated coordinates, and then**
  3. **Scale the rotated coordinates to complete the composite transformation.**
- To shorten this process, we have to use 3×3 transformation matrix instead of 2×2 transformation matrix. **To convert a 2×2 matrix to 3×3 matrix, we have to add an extra dummy coordinate W.**

# If you want to move (translate) a point like (2, 3, 4) by (1, 0, -2), you'd have to do this:

New x = 2 + 1 = 3
New y = 3 + 0 = 3
New z = 4 - 2 = 2

**That's simple math, but you can't do it with a matrix alone unless you use homogeneous coordinates.**

**In homogeneous coordinates, (2, 3, 4) becomes (2, 3, 4, 1)**
**Now you can use a single 4×4 matrix like this:**

$$\begin{vmatrix} 1 & 0 & 0 & Tx \\ 0 & 1 & 0 & Ty \\ 0 & 0 & 1 & Tz \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Imagine standing on a road - the farther it goes, the closer the edges look together. That's perspective.

With homogeneous coordinates, OpenGL can divide x, y, and z by w to shrink far objects and stretch close ones:

ProjectedX = x / w
ProjectedY = y / w
ProjectedZ = z / w

a nearby point = (2, 2, 2, 1) → looks normal
a far point = (2, 2, 2, 10) → gets divided → (0.2, 0.2, 0.2)

# Homogenous Coordinates

Conversion Between Cartesian and Homogeneous Coordinates. To convert from homogeneous coordinates to Cartesian coordinates, you can divide the first three components by the last component $w$.

A point $(x,y)$ in Cartesian coordinates is represented as:$(x,y) \rightarrow (wx,wy,w)$ where $w \neq 0$. If $w=1$, the point is represented as $(x,y,1)$.

- To represent points and transformations using homogeneous coordinates, you extend the traditional Cartesian coordinate system by **adding an additional dimension.**
- Here's a breakdown of how to represent homogeneous coordinates in both 2D and 3D systems:

**In 2D, a point in Cartesian coordinates $(x,y)$ is represented in homogeneous coordinates as:$(x,y) \rightarrow (x,y,w)$**

**Where:**

**$x$ and $y$ are the original Cartesian coordinates.**

**$w$ is a non-zero scalar value that allows for the representation of points at infinity when set to 0.**

**Commonly, $w$ is set to 1 for finite points.**

*Examples: Finite Point: The point (2,3) in Cartesian coordinates can be represented as:(2,3,1)*

# Advantages of Using Homogeneous Coordinates

- **Unified Representation**: All transformations, including translation, scaling, rotation, and shearing, can be represented uniformly using **matrix multiplications.**
- **Single Matrix Multiplication**: You can apply a series of transformations with a single matrix multiplication, simplifying the computational process.
- **Perspective Projection**: Easily handle perspective projections, enabling depth representation and vanishing points.
- **Efficiency**: More efficient computations in graphics rendering, especially for real-time applications.
- **Simpler Interpolations**: Allows for smooth transitions and easier implementation of key-frame animations.

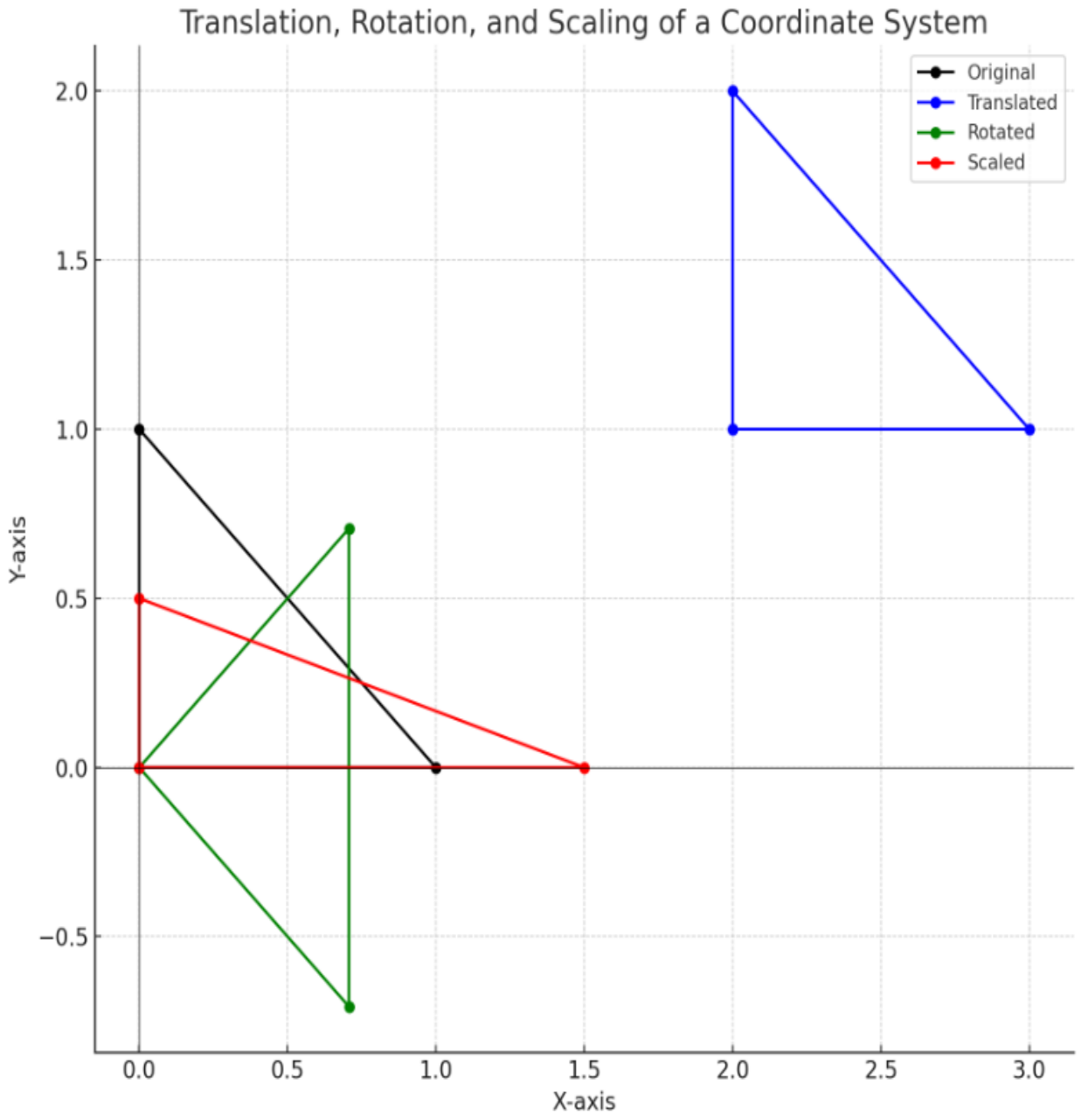
**Even though w looks weird, computers divide everything by w at the end so we can still draw it on your screen. It's just a smart trick to make drawing faster and cooler.**

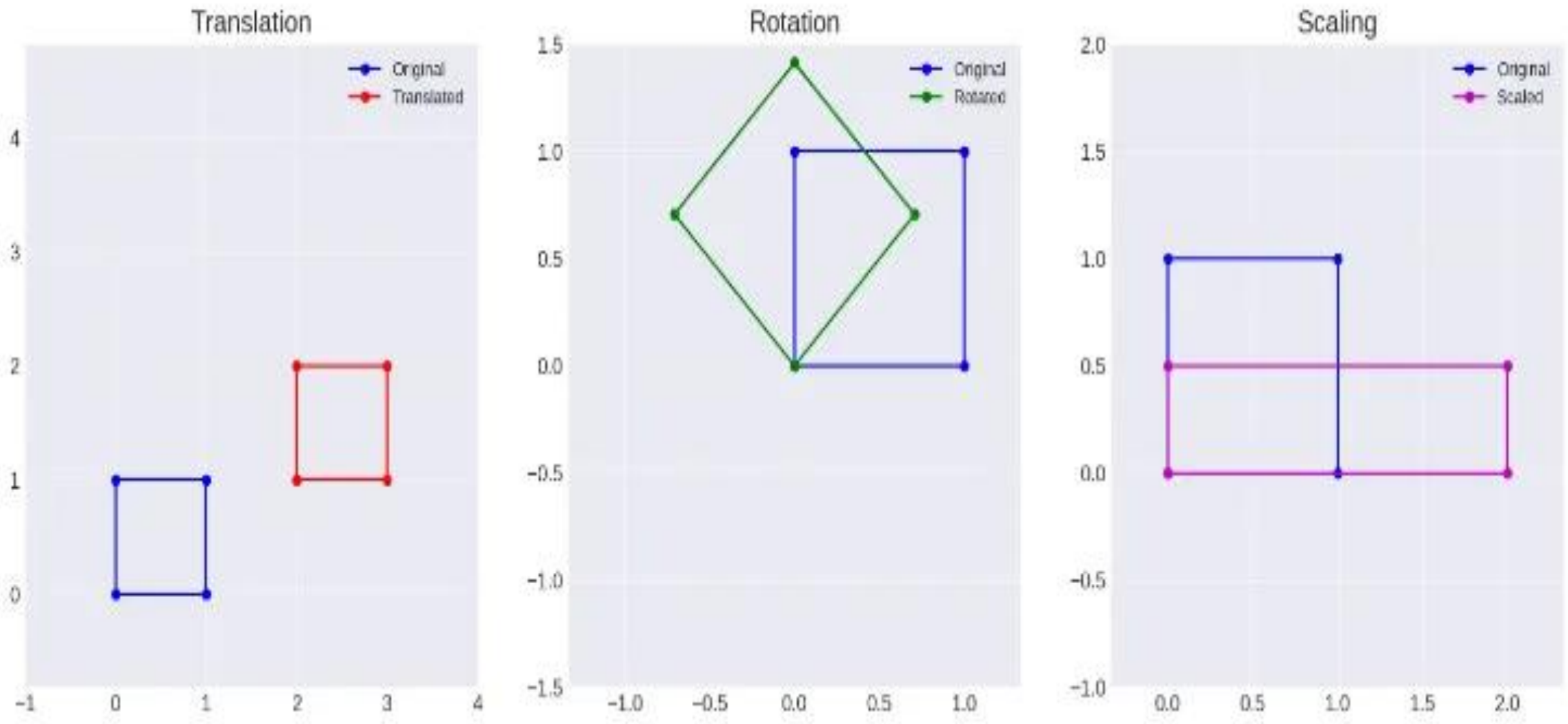
# Geometrical Transformations

Geometrical transformations are fundamental to computer graphics as they allow the manipulation of objects in a scene, such as moving, scaling, rotating, or skewing objects.

**Now, let's dive into matrices. Think of matrices as a grid of numbers arranged in rows and columns.**

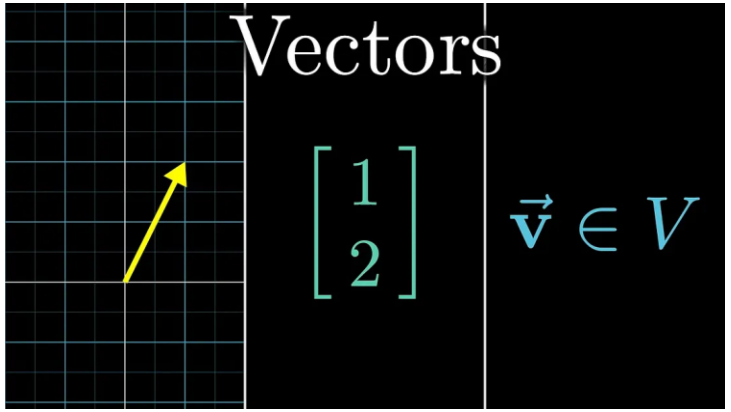
A transformation matrix is a special type of matrix used to perform operations such as shifting, rotating, or scaling objects in a coordinate system.




Translation
Original
Translated
4
3
2
1
0
−1
0
1
2
3
4
Rotation
Original
Rotated
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
Scaling
Original
Scaled
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
0.0
0.5
1.0
1.5
2.0

# Linear Algebra – Vectors and Matrices

- Understanding vectors and matrices is fundamental for representing and processing image data.
  - **Vectors: In computer vision, a vector is a one-dimensional array of values.**
  - **Vectors can represent various types of data, including pixel intensities, image coordinates, feature descriptors, and more.**
  - **Matrices are two-dimensional arrays of values, consisting of rows and columns.**
- Operations like addition, subtraction, multiplication, and transformation are essential for image manipulation.




**Vectors tell us:**
A position (where something is)
A movement (where something is going)

The matrix and the vector work together to transform the object

# Matrix



- Perhaps the most universal tools of graphics programs are the matrices that change or transform points and vectors.
- The identity matrix contains all zeros, with ones along the diagonal:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

*Matrix addition* is straightforward:

$$\begin{bmatrix} A+a & B+b & C+c & D+d \\ E+e & F+f & G+g & H+h \\ I+i & J+j & K+k & L+l \\ M+m & N+n & O+o & P+p \end{bmatrix} = \begin{bmatrix} A & B & C & D \\ E & F & G & H \\ I & J & K & L \\ M & N & O & P \end{bmatrix} + \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix}$$

The **identity matrix** is a special matrix that does… **nothing!**

**Any point or matrix multiplied by the identity matrix is unchanged.**

Imagine you have a magic mirror. You show it an object, and it reflects exactly the same thing — no change. That's what the identity matrix does!

# Matrix

Matrix multiplication is a bit trickier. You **multiply rows of the first matrix** by **columns of the second matrix**, and then add the results. For matrix multiplication to work, the number of columns in the first matrix must match the number of rows in the second matrix.

***In 3D graphics, multiplying a point by a matrix*** is in most cases done right to left, and produces a point, as follows:

$$\begin{pmatrix} AX+BY+CZ+D \\ EX+FY+GZ+H \\ IX+JY+KZ+L \\ MX+NY+OZ+P \end{pmatrix} = \begin{bmatrix} A & B & C & D \\ E & F & G & H \\ I & J & K & L \\ M & N & O & P \end{bmatrix} * \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}$$

**Example:**

If $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$, you multiply them as follows:

To get the element at row 1, column 1 of the result, multiply the first row of $A$ by the first column of $B$:

$$(1 \times 5) + (2 \times 7) = 5 + 14 = 19$$

To get the element at row 1, column 2, multiply the first row of $A$ by the second column of $B$:

$$(1 \times 6) + (2 \times 8) = 6 + 16 = 22$$

***Multiplying a 4x4 Matrix by another 4x4 matrix*** is done as follows:

$$\begin{bmatrix} A & B & C & D \\ E & F & G & H \\ I & J & K & L \\ M & N & O & P \end{bmatrix} * \begin{bmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{bmatrix} =$$

$$\begin{bmatrix} Aa+Be+Ci+Dm & Ab+Bf+Cj+Dn & Ac+Bg+Ck+Do & Ad+Bh+Cl+Dp \\ Ea+Fe+Gi+Hm & Eb+Ff+Gj+Hn & Ec+Fg+Gk+Ho & Ed+Fh+Gl+Hp \\ Ia+Je+Ki+Lm & Ib+Jf+Kj+Ln & Ic+Jg+Kk+Lo & Id+Jh+Kl+Lp \\ Ma+Ne+Oi+Pm & Mb+Nf+Oj+Pn & Mc+Ng+Ok+Po & Md+Nh+Ol+Pp \end{bmatrix}$$

# Matrix * Matrix

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, B = \begin{bmatrix} x & y \\ z & w \end{bmatrix}$$

$$A * B = \begin{bmatrix} ax + bz & ay + bw \\ cx + dz & cy + dw \end{bmatrix}$$

$$? = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Does A*B = B*A? NO

What does the identity do?

AI=A

To multiply two matrices, the number of columns in the first matrix must equal the number of rows in the second matrix. If: Matrix $A$ has dimensions $m \times n$ (m rows and n columns).Matrix $B$ has dimensions $n \times p$ (n rows and p columns). The resulting matrix $C$ will have dimensions $m \times p$:

# Identity Matrix



- An identity matrix is a special type of square matrix that plays a **similar role to the number 1 in arithmetic.**
- An identity matrix is a square matrix (having the same number of rows and columns), in which all the elements of the main diagonal (from the top-left to the bottom-right) are 1, and all other elements are 0.

$$I_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

**Multiplicative Identity**: When any matrix A of size n×n is multiplied by the identity matrix of the same size In, the matrix A remains unchanged.

# Transformation Matrices



- Transformation matrices are typically used with **homogeneous coordinates,** which enable **uniform treatment of all transformations**, including translation, that wouldn't otherwise be possible using traditional Cartesian coordinates.

$$M = \begin{bmatrix} m_{11} & m_{12} & m_{13} & tx \\ m_{21} & m_{22} & m_{23} & ty \\ m_{31} & m_{32} & m_{33} & tz \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

**Where:**

- **The upper-left 3x3 part represents rotation and scaling.**
- **The right-most column represents translation.**
- **The bottom row handles perspective transformations (usually 0, 0, 0, 1 for standard transformations).**

# Types of Transformations



1. **Translation**: Moving an object without altering its shape.
2. **Rotation**: Rotating an object around a pivot point.
3. **Scaling**: Changing the size of an object while maintaining its proportions.
4. **Reflection**: Flipping an object over a specified axis.
5. **Shearing**: Distorting an object along a specific direction.

2D Translation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad \mathbf{P}' = \mathbf{T}\left(t_x, t_y\right) \cdot \mathbf{P}$$

2D Rotation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad \mathbf{P}' = \mathbf{R}\left(\theta\right) \cdot \mathbf{P}$$

2D Scaling

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad \mathbf{P}' = \mathbf{S}\left(S_x, S_y\right) \cdot \mathbf{P}$$

# Why Matrix Multiplication?



- While it may seem that adding transformation values could be a simpler approach, using matrix multiplication for transformations offers significant advantages, particularly in computer graphics and linear algebra.
- Here are the key reasons why we use matrices for transformations instead of simple addition:
  1. **Combining Transformations**
  2. **Preserve the correct order**
  3. **Unified Representation**
  4. **Single Calculation**

finalMatrix = scaleMatrix × rotateMatrix × translateMatrix;

newPosition = finalMatrix × originalPosition;

**Matrix multiplication lets us combine multiple transformations into one.**
Instead of doing:
- Move right →
- Then rotate →
- Then scale →

You can create **one big transformation matrix** that does **all at once**!

# Translation

**Translation involves shifting an object from one location to another without changing its shape, size, or orientation.**

- In normal Cartesian coordinates, translation (moving an object by a certain amount) can't be easily represented by matrix multiplication.
- However, in homogeneous coordinates, it becomes straightforward. For example, if we want to translate a point (x, y) by (tx, ty):

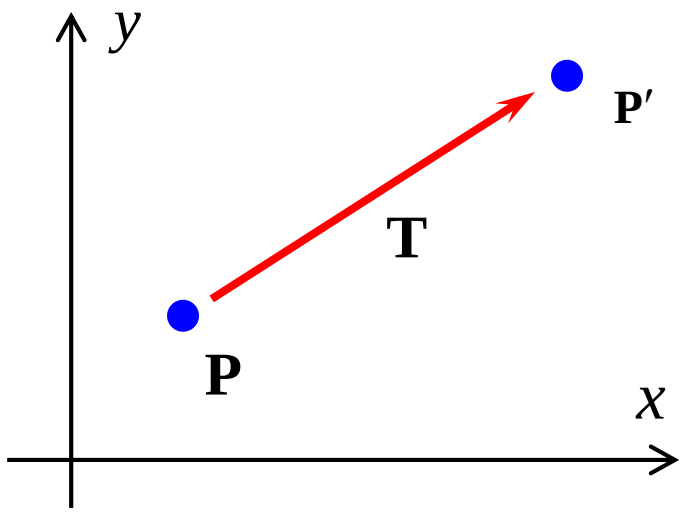
$$\begin{pmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + tx \\ y + ty \\ 1 \end{pmatrix}$$

# Translation

$$x' = x + t_x, \quad y' = y + t_y$$

$$\mathbf{P} = \begin{bmatrix} x \\ y \end{bmatrix}, \quad \mathbf{P}' = \begin{bmatrix} x' \\ y' \end{bmatrix}, \quad \mathbf{T} = \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

April 2010




$$\mathbf{P}' = \mathbf{P} + \mathbf{T}$$

$$T(tx, ty) = \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix}$$

To translate a point $(x, y)$, multiply the point by the translation matrix:

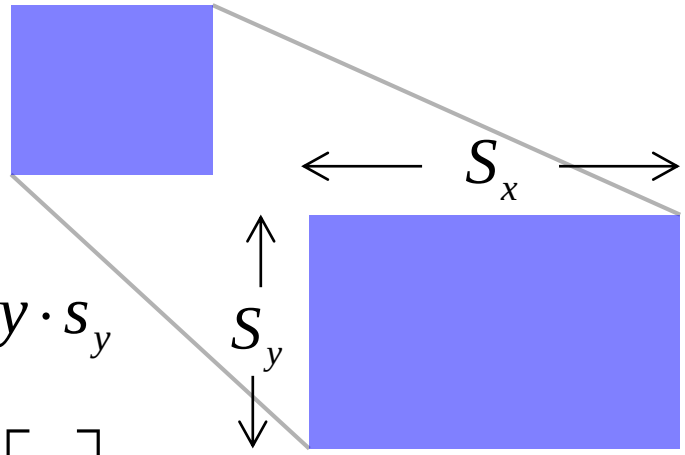
$$P' = T(tx, ty) \cdot P_h = \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + tx \\ y + ty \\ 1 \end{bmatrix}$$

Translation matrix is used to move items from one location to another.

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & T_x \\ 0 & 1 & 0 & T_y \\ 0 & 0 & 1 & T_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

# Scaling

Scaling alters the size of an object while maintaining its proportions. It can be uniform (same scale factor for both axes) or non-uniform (different scale factors for each axis).



$$x' = x \cdot s_x, \quad y' = y \cdot s_y$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

$$\mathbf{P}' = \mathbf{S} \cdot \mathbf{P}$$

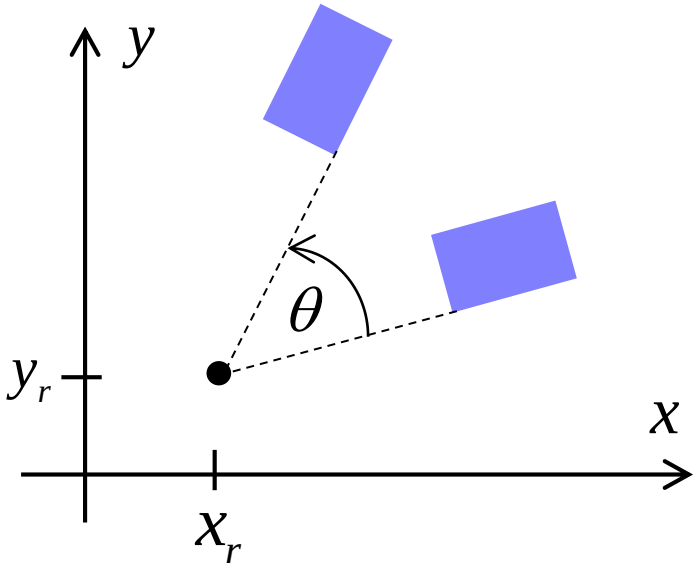
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 & 0 \\ 0 & S_y & 0 & 0 \\ 0 & 0 & S_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

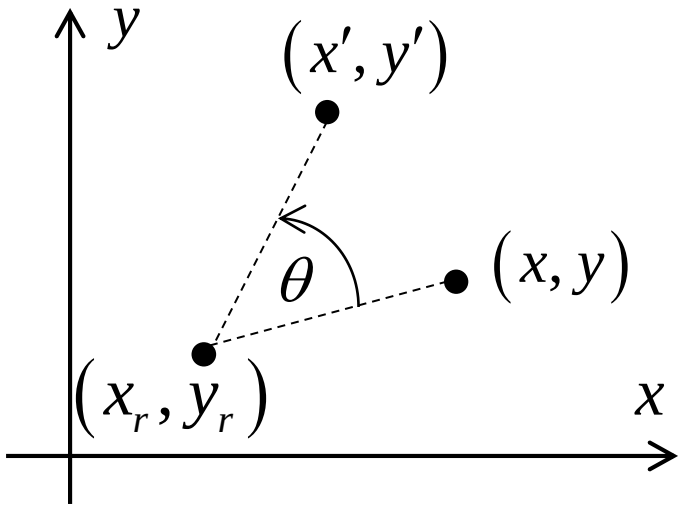
$$S(sx, sy) = \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Applying this to a point $(x, y)$:

$$P' = S(sx, sy) \cdot P_h = \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} sx \cdot x \\ sy \cdot y \\ 1 \end{bmatrix}$$

# 2D Rotation



$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

To rotate a point $(x, y)$:

$$P' = R(\theta) \cdot P_h = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

Where:

$$x' = x \cdot \cos\theta - y \cdot \sin\theta$$

$$y' = x \cdot \sin\theta + y \cdot \cos\theta$$

$$\mathbf{R} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

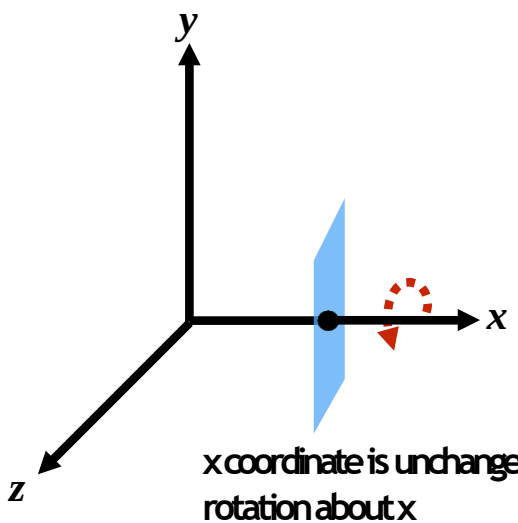
Rotation in angle $\theta$ about a pivot (rotation) point $(x_r, y_r)$.

# Rotations in 3D

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) & 0 \\ 0 & \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

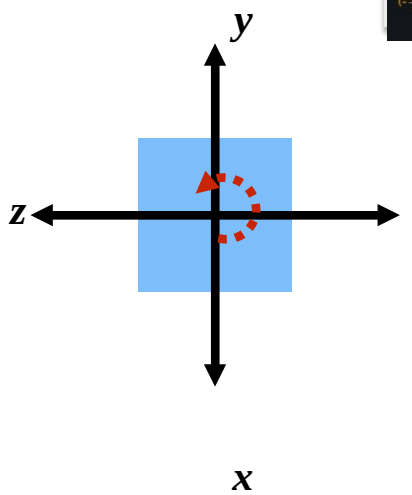
$$R_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

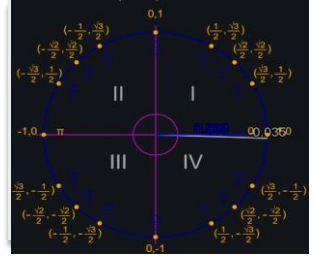
$$R_z(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 & 0 \\ \sin(\theta) & \cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

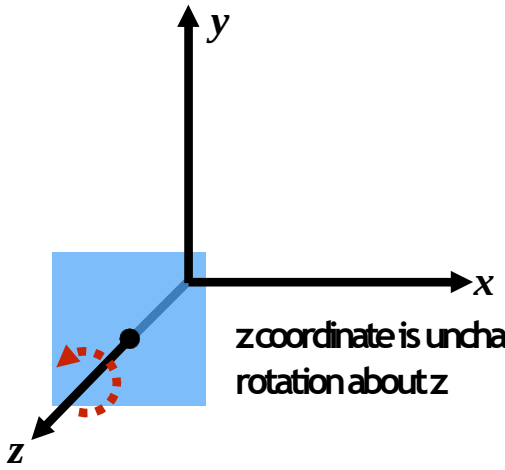



x coordinate is unchanged by rotation about x

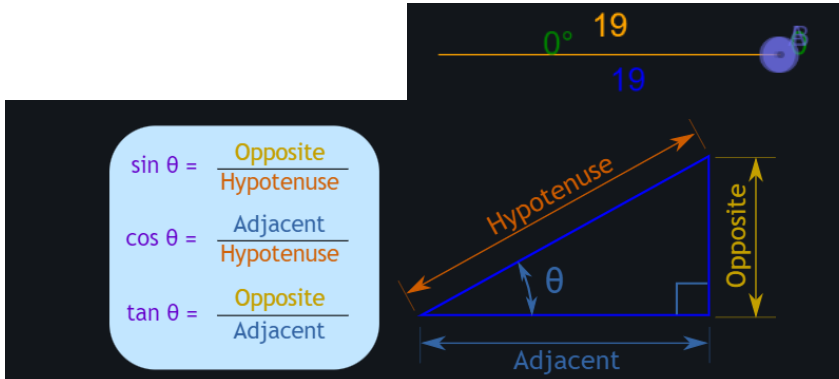
View looking down -x axis:




z coordinate is unchanged by rotation about z




For a given angle $\theta$ each ratio stays the same no matter how big or small the triangle is

**2. Rotation about the X-axis**

- **Counterclockwise (CCW):**

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) & 0 \\ 0 & \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Clockwise (CW):**

$$R_x(-\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(-\theta) & -\sin(-\theta) & 0 \\ 0 & \sin(-\theta) & \cos(-\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This simplifies to:

$$R_x(-\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & \sin(\theta) & 0 \\ 0 & -\sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

**3. Rotation about the Y-axis**

- **Counterclockwise (CCW):**

$$R_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Clockwise (CW):**

$$R_y(-\theta) = \begin{bmatrix} \cos(-\theta) & 0 & \sin(-\theta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(-\theta) & 0 & \cos(-\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This simplifies to:

$$R_y(-\theta) = \begin{bmatrix} \cos(\theta) & 0 & -\sin(\theta) & 0 \\ 0 & 1 & 0 & 0 \\ \sin(\theta) & 0 & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

**1. Rotation about the Z-axis**

- **Counterclockwise (CCW):**

$$R_z(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 & 0 \\ \sin(\theta) & \cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Clockwise (CW):**

$$R_z(-\theta) = \begin{bmatrix} \cos(-\theta) & -\sin(-\theta) & 0 & 0 \\ \sin(-\theta) & \cos(-\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This simplifies to:

$$R_z(-\theta) = \begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 & 0 \\ -\sin(\theta) & \cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

# Reflection

(left) Original Image (right) Image after horizontal reflection.

- **Reflection** is a vital geometric transformation in both 2D and 3D space, allowing for the mirroring of points over specific lines or planes.
  - The reflection of a point (x, y) over the **x-axis results in the point (x,−y).**
  - The reflection of a point $(x,y)$ over the **y-axis results in the point $(-x,y)$.**
  - The reflection of a point $(x,y)$ over the line $y=x$ **results in the point $(y,x)$.**

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R_x \cdot \begin{bmatrix} 3 \\ 4 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ -4 \\ 1 \end{bmatrix}$$

$$R_y = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R_y \cdot \begin{bmatrix} 3 \\ 4 \\ 1 \end{bmatrix} = \begin{bmatrix} -3 \\ 4 \\ 1 \end{bmatrix}$$

$$R_{y=x} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R_{y=x} \cdot \begin{bmatrix} 3 \\ 4 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \\ 1 \end{bmatrix}$$

# Reflection in 3D

- Using Homogeneous Coordinates In 3D, a point $(x,y,z)$ is represented as $(x,y,z,1)$.
  1. **Reflection over the XY-plane:** The reflection over the XY-plane transforms $(x,y,z)$ to $(x,y,-z)$.
  2. **Reflection over the XZ-plane:** The reflection over the XZ-plane transforms $(x,y,z)$ to $(x,-y,z)$.
  3. **Reflection over the YZ-plane:** The reflection over the YZ-plane transforms $(x,y,z)$ to $(-x,y,z)$.

$$R_{XY} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_{XY} \cdot \begin{bmatrix} 3 \\ 4 \\ 5 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \\ -5 \\ 1 \end{bmatrix}$$

$$R_{XZ} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_{XZ} \cdot \begin{bmatrix} 3 \\ 4 \\ 5 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ -4 \\ 5 \\ 1 \end{bmatrix}$$

$$R_{YZ} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_{YZ} \cdot \begin{bmatrix} 3 \\ 4 \\ 5 \\ 1 \end{bmatrix} = \begin{bmatrix} -3 \\ 4 \\ 5 \\ 1 \end{bmatrix}$$

# Shearing



- Shearing is a transformation that shifts each point of an object by a **distance proportional to its distance from a specific axis.**
- The result is a slanted version of the object, while preserving its area and parallelism of lines.
- In 2D, shearing transforms points by shifting them along the x-axis or y-axis, depending on the shear factor.

$$S_x(k_x) = \begin{bmatrix} 1 & k_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$S_y(k_y) = \begin{bmatrix} 1 & 0 & 0 \\ k_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

# Transformation in Practice: OpenGL

- glTranslatef(tx, ty, tz); // tx, ty, and tz are translation amounts
- glScalef(sx, sy, sz); // sx, sy, and sz are scaling factors
- glRotatef(angle, x, y, z);  // glRotatef(45.0, 0.0, 0.0, 1.0);

```
GLfloat reflectionMatrixY[16] = {
    -1.0, 0.0, 0.0, 0.0,
    0.0, 1.0, 0.0, 0.0,
    0.0, 0.0, 1.0, 0.0,
    0.0, 0.0, 0.0, 1.0
};

glMultMatrixf(reflectionMatrixY);  // Apply the reflection matrix
```

```
GLfloat shearMatrix[16] = {
    1.0, kx, 0.0, 0.0,
    0.0, 1.0, 0.0, 0.0,
    0.0, 0.0, 1.0, 0.0,
    0.0, 0.0, 0.0, 1.0
};

glMultMatrixf(shearMatrix); // Applies the custom shear matrix
```

```
// Set the camera position
gluLookAt(0.0, 0.0, 5.0, // eye position
    0.0, 0.0, 0.0, // center of the scene
    0.0, 1.0, 0.0); // up vector

// ---- Transform and draw the first object (Square) ----
glPushMatrix(); // Save the current matrix

// Apply transformation for the square
glTranslatef(-1.5f, 0.0f, 0.0f); // Move left
glRotatef(45, 0.0f, 0.0f, 1.0f); // Rotate 45 degrees clockwise around Z-axis
glScalef(0.5f, 0.5f, 1.0f); // Scale down

// Draw the square (as two triangles)
glBegin(GL_QUADS);
glVertex3f(-1.0f, -1.0f, 0.0f);
glVertex3f(1.0f, -1.0f, 0.0f);
glVertex3f(1.0f, 1.0f, 0.0f);
glVertex3f(-1.0f, 1.0f, 0.0f);
glEnd();

glPopMatrix(); // Restore the matrix to the state before transformations
```

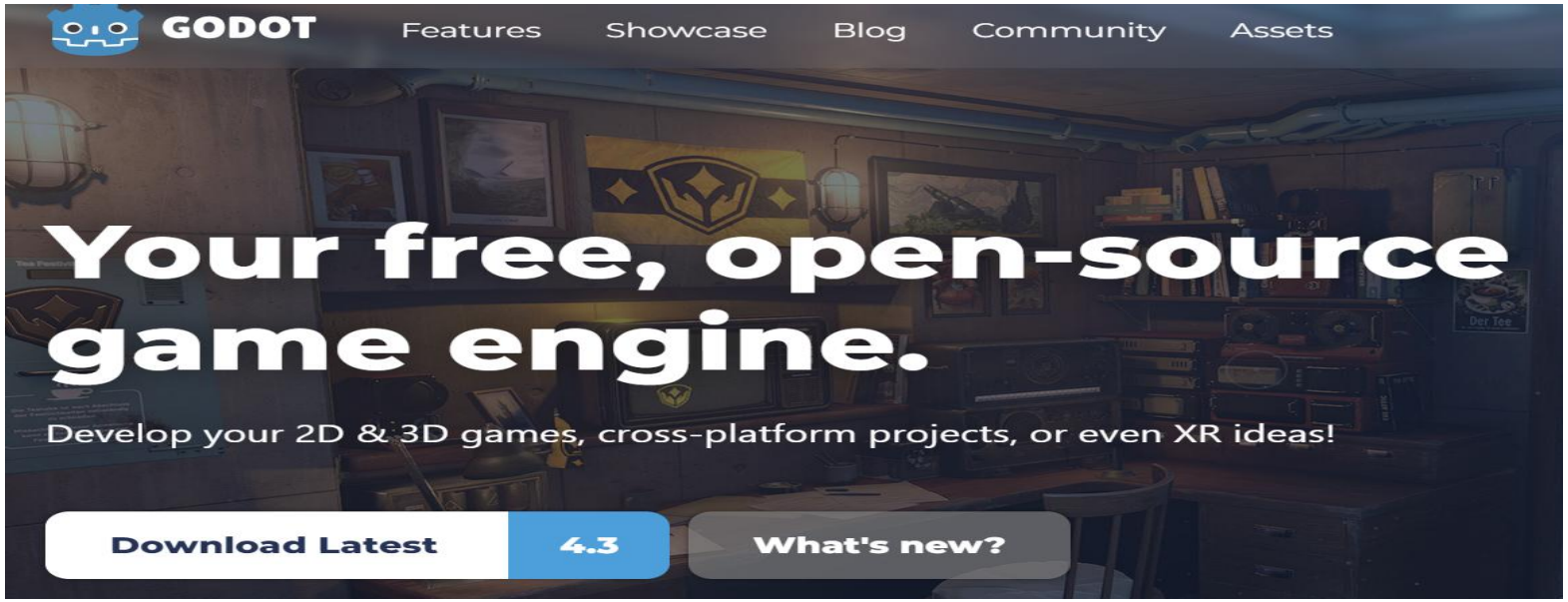
Parameters:

- eyeX, eyeY, eyeZ: The position of the camera or the viewer in 3D space (the eye point).
- centerX, centerY, centerZ: The point in 3D space that the camera is looking at (the target point or center).
- upX, upY, upZ: The direction that defines "up" for the camera. This is typically the positive Y-axis (0, 1, 0) but can vary depending on the desired orientation.

# Transforms in Professional Software



- Godot: https://godotengine.org/
- Unity: https://unity.com/
- Unreal: https://www.unrealengine.com/en-US
- Blender: https://www.blender.org/
- GLM: https://github.com/g-truc/glm



# Classroom Exercise



1. Given a point $P(3,4)$, translate it by $tx$=5 and $ty$=−2. What is the new position of the point, and its representation in HC?
2. Scale a point $P(2,3)$ by factors $sx$=2 and $sy$=3.What is the new position of the point and its representation in HC?.
3. Rotate the point $P(1,0)$ by 90° counterclockwise about the origin. What is the new position of the point.
4. Apply an x-shear with a shear factor of 1.5 to the point $P(3,2)$. What is the new position of the point?
5. Reflect the point $P(2,-3)$ about the x-axis. What is the new position of the point?